AwarenessAlert: A Crowdsourced Food and Fuel Safety Reporting Platform for Dhaka

By

Zearin Akhter 0432320005101014

Department of CSE

A Project Report Submitted in Partial Fulfillment of the Requirements of the Course
CSE416 – Software Project Design and Development
Semester: Autumn 2026 | Short Format

Course Instructor

AL Imtiaz

Assistant Professor

Department of CSE



University of Information Technology and Sciences (UITS)
Baridhara J Block, Dhaka 1212
October, 2026

Project at a Glance

Item	Details
Repository URL	https://github.com/Zearin14/Zearin_CSE-416_SDD_Project-_AwarenessAlert
Final release tag	v0.9-test (the version tested and presented)
Commit SHA of the release	01fec35a7d11b50df3a0fa4b0a92b378b1d21e60
Deployed product URL	https://awarenessalert.infinityfreeapp.com (InfinityFree free hosting, PHP 8.4 + MySQL; live since 31 Oct 2026)
Demo / test account	admin@awarenessalert.test / Admin#1234 (admin) dncrp@awarenessalert.test / Org#1234 (authority) Test accounts with demo data only
Demo video (optional)	Not recorded
Presentation slides (Week 13)	0432320005101014_Exp13.pdf (Experiment 13)
Technology stack	React 19 + TypeScript (Vite, Tailwind CSS 4); PHP 8 REST API with PDO; MySQL / MariaDB; GitHub Actions
Test summary	219 / 219 automated test runs pass; 100% line and branch coverage of the core logic; 0 open S1 / S2 defects

AI Tool Use Acknowledgement

We acknowledge that the assistant tools listed below were used in this project. In every case we first produced our own solution, used the tool only to refine, explore or verify it, and checked every accepted output ourselves. We understand all submitted work, including every tool-assisted part, and can explain and reproduce it. Each use is recorded in detail in the Tool Usage Log (Appendix A).

Summary of assistant tools used

Tool (name and version)	Used for (phase / task)	Extent of use	Verified by (member)
Claude (Anthropic AI assistant)	Testing: test tooling, CI file and test code from my SRS-based test cases; Documentation: drafting the test report, this report and the slides from my SRS, code and test results	Substantial	Zearin Akhter
Qodo (Qodo Merge PR review, free plan)	Code review: automatic review of pull request #1	Minor	Zearin Akhter
Figma Make	Design and coding: React + Vite + Tailwind project scaffold and first UI screens	Moderate	Zearin Akhter

Claude was used substantially. The requirements (SRS), the trust rule, the moderation workflow and every expected test value came from my own SRS; I checked every number in the reports against the actual test runs and can explain and reproduce all of it (details in Appendix A).

Use in writing this report: ☐ No assistant tool was used to write this report ☐ A tool was used only to polish language ☒ A tool (Claude) helped draft parts of the text from my own material; I reviewed and corrected all technical content

Member (name, ID)	Signature	Date
Zearin Akhter (0432320005101014)	Zearin	31-10-2026

Justification Report

Below is the CO-PO Mapping and Complex Engineering Problem (CEP) Justification Report for the project titled "AwarenessAlert: A Crowdsourced Food and Fuel Safety Reporting Platform for Dhaka", developed for the course CSE416 Software Project Design and Development (Autumn 2026).

CO-PO Mapping

CO No.	Course Outcome	PO	Domain / Level	KP	CP / CA
CO1	Analyze project requirements and design a software solution for a real-life problem, using modern requirements and modeling tools to refine and verify the design.	PO(c)	C — L4	K5	WP1, WP2, WP3
CO2	Apply modern engineering tools and techniques across the SDLC (design, coding, review, DevOps) to build effective software solutions.	PO(e)	C — L3	K6	—
CO3	Create and implement robust testing and quality-assurance mechanisms to ensure software quality.	PO(i)	C — L5	K6	WP1, WP2, WP3
CO4	Implement, deploy and manage a real-life complex software product end-to-end, with version control, CI/CD and release management.	PO(k)	P — L6 / P5	—	EA1, EA2, EA3, EA5
CO5	Investigate a real-life problem by evaluating alternative solutions and justifying design decisions with evidence.	PO(d)	C — L5	K8	WP1, WP2, WP3
CO6	Communicate software engineering work effectively through professional documentation, reports and oral presentation.	PO(g)	C/A — L4 / A3	K7	EA1, EA2
CO7	Function effectively as a member of a two-person team, sharing responsibility and integrating individual contributions.	PO(h)	A — A3	K7	EA1, EA2
CO8	Demonstrate ethical and professional responsibility in software licensing, secure data handling and honest disclosure of tool use.	PO(f)	A — A3	K7	EA4

Domain: C = Cognitive (Bloom L1–L6), P = Psychomotor (P1–P7), A = Affective (A1–A5). KP = Knowledge Profile (K1–K8). CP = Complex Problem attributes (WP1–WP7). CA = Complex Activity attributes (EA1–EA5).

Mapping of COs with Program Outcomes (POs)

CO	(a)	(b)	(c)	(d)	(e)	(f)	(g)	(h)	(i)	(j)	(k)	(l)
CO1			**3**									
CO2					**3**							
CO3									**3**			
CO4											**3**	
CO5				**2**								
CO6							**2**					
CO7								**2**				
CO8						**2**						

Correlation strength: 3 = High, 2 = Medium, 1 = Low.

Justification of PO through COs

Note – During justification you have to mention where you have added the relevant data/information in your report.

CO	PO	Justification	Where in this report
CO1	**PO(c)**	The scattered, unpublished complaints about food and fuel adulteration in Dhaka were analysed and turned into an SRS with 66 functional requirements, then into use-case, ER, class, sequence, activity and DFD models. Every requirement traces to design, code and tests (Table 4.1).	*Ch. 2 (2.2–2.3); Table 4.1*
CO2	**PO(e)**	Modern tools were used in each phase: draw.io for UML, React/TypeScript/Vite and PHP for coding, Qodo for pull-request review, Vitest, Playwright, axe and Lighthouse for testing, GitHub Actions for CI. My SRS and design came first; tools refined and verified them.	*Ch. 2 (2.3); Ch. 3; Ch. 4 (4.3); App. A*
CO3	**PO(i)**	A test strategy with unit, integration and end-to-end levels, equivalence partitioning, boundary values and state transitions gave 219 passing runs, 100% core coverage and an 86.9% mutation score. Code review, npm audit and gitleaks covered quality and security.	*Ch. 4 (4.1–4.2); App. C*
CO4	**PO(k)**	The product was built on a feature branch, reviewed through a pull request, integrated with a GitHub Actions pipeline and tagged as v0.9-test. Plan versus actual, effort estimate and risks are in Table 4.3. The product is deployed on InfinityFree free hosting at https://awarenessalert.infinityfreeapp.com.	*Ch. 3; Ch. 4 (4.3–4.4); Project at a Glance*
CO5	**PO(d)**	Two existing solutions (BFSA hotline, Google Maps reviews) and alternative technologies were compared; the three-source trust rule and server-side state machine were chosen and justified with test evidence.	*Ch. 2 (2.1, 2.4)*

CO	PO	Justification	Where in this report
CO6	**PO(g)**	SRS, design diagrams, README, the Software Test Report, this report and the final presentation document the work.	*Whole report; Ch. 4 (4.3); App. C*
CO7	**PO(h)**	Individual project: all modules, tests and documents were produced by one member; the commit and pull-request history is the evidence.	*Ch. 1 (1.3); App. B (B.1–B.3)*
CO8	**PO(f)**	Open-source licences (MIT, Apache-2.0, PHP License, GPL v2) are respected; passwords are hashed, IP and device data are admin-only, secrets are kept out of code, and all assistant-tool use is disclosed.	*AI Tool Use Acknowledgement; Ch. 4 (4.2); Ch. 5 (5.2); App. A*

Complex Engineering Problem Justification

Note – During justification you have to mention where you have added the relevant data/information in your report.

1. Mandatory Attribute

Attribute	Linked COs	Justification
WP1: Depth of Knowledge Required	CO1, CO3, CO5	The trust rule needs engineering design knowledge (K5): weighted scoring, time windows, independent-source counting and a state machine for moderation, plus knowledge of consumer-protection practice in Bangladesh (K8). See Ch. 2 (2.3–2.4) and 4.1.

2. Supporting Attributes

Attribute	Linked COs	Justification
WP2: Range of Conflicting Requirements	CO1, CO3, CO5	Openness (anyone may report without an account) conflicts with fairness to vendors (one fake report must not hurt them) and with privacy (reporters must stay anonymous while abuse is still detectable). Resolved by the three-source rule, admin-only IP/device data and moderation (2.4).
WP3: Depth of Analysis Required	CO1, CO3, CO5	Alternatives were compared (Table 2.1, 2.4); risks and security were analysed (4.2, Table 4.3); mutation testing and planted bugs checked that the tests really detect faults (4.1).

Range of Complex Engineering Activities (CEA) Justification

Attribute	Linked COs	Justification
EA1: Range of Resources	CO4, CO6, CO7	People: one developer, instructor feedback; tools: React, PHP, MariaDB, Vitest, Playwright, GitHub Actions, Qodo; information: SRS, BFSA and DNCRP sources; about 14 weeks.
EA2: Level of Interaction	CO4, CO6, CO7	Needs of shoppers, vendors, moderators and authorities (BSTI, DNCRP) had to be balanced with platform limits such as no live government API and free shared hosting.

Attribute	Linked COs	Justification
EA3: Innovation	CO4	A weighted, multi-source trust score combined with an official authority-outcome step, instead of a simple complaint list.
EA4: Consequences for Society and the Environment	CO8	Privacy of reporters, fairness to vendors, accessibility (WCAG checks), secure handling of data and licences — Ch. 4.2 and 5.2.
EA5: Familiarity	CO4, CO5	Principles-based work beyond routine CRUD: state-machine design, mutation testing and traceability to an SRS.

Mapping to Communication (CO6):

- **Written Technical Documents: this report, the SRS (Experiment 2), design document (Experiment 3), Software Test Report (Experiment 9) and the README.**
- **Oral Presentations: Week 13 presentation and viva with a live demonstration (0432320005101014_AwarenessAlert.pdf).**
- **Visual Representation: use-case, ER, sequence and architecture diagrams (Figs. 2.1–2.4) and product screenshots (Figs. 4.1–4.4).**

Abstract

Food and fuel adulteration is common in Dhaka, but warnings stay private: complaints go to hotlines and the public never sees them. AwarenessAlert is a web platform where anyone can report a suspected safety problem at a shop or filling station without creating an account. Moderators verify the reports, a routing rule sends verified reports to the responsible authority (BSTI, DNCRP or an accredited laboratory), and the authority records an official outcome that is shown publicly. A vendor's trust status on the public map changes only when at least three independent sources report within 30 days or an authority confirms a problem, so a single fake complaint cannot harm an honest vendor. The system was built with React and TypeScript, a PHP REST API and MySQL. It was tested at unit, integration and end-to-end level: 219 automated test runs pass, all 66 functional requirements are traced to tests, and the core logic has 100% line and branch coverage. The product is live at https://awarenessalert.infinityfreeapp.com.

Keywords: food safety, crowdsourcing, trust score, web application, software testing

Table of Contents

Abstract vii

Chapter 1 Introduction 1

1.1 Problem and Motivation 1

1.2 Objectives and Scope 1

1.3 Team Organization 2

Chapter 2 Requirements and Design 3

2.1 Existing Solutions and Alternatives 3

2.2 Requirements Summary 3

2.3 System Design 4

2.4 Design Decisions and Rationale 6

Chapter 3 Implementation 7

3.1 Implementation and Integration 7

3.2 Tool-Assisted Refinement 7

Chapter 4 Testing, Deployment and Project Management 9

4.1 Testing and Quality Assurance 9

4.2 Code Review, Security and Ethics 10

4.3 Documentation, CI/CD and Deployment 10

4.4 Project Management 10

4.5 Results and Discussion 11

Chapter 5 Conclusion 14

5.1 Conclusion and Future Work 14

5.2 Reflection 14

References 15

Appendix A: Tool Usage Log 16

Appendix B: Team Contribution and Peer Evaluation..... 17

Appendix C: Supporting Documents 18

List of Figures

Fig. 2.1 System architecture 4
Fig. 2.2 Use case diagram 5
Fig. 2.3 Entity-relationship diagram 6
Fig. 2.4 Sequence diagram: report to authority outcome 6
Fig. 4.1 Public safety map 12
Fig. 4.2 Report form 12
Fig. 4.3 Vendor profile with explained trust score 13
Fig. 4.4 Authority portal: recording an outcome 13

List of Tables

Table 2.1 Comparison with existing solutions 3
Table 4.1 Requirements traceability: SRS to design, code and tests 9
Table 4.2 Test and quality summary 9
Table 4.3 Planned versus actual schedule and key risks 10
Table A.1 Tool usage log 16
Table B.1 Contribution summary 17
Table B.2 Agreed contribution split 17
Table B.3 Peer evaluation 17
Table E.1 Report evaluation 19

Abbreviations and Symbols

CSE	Computer Science and Engineering
SDLC	Software Development Life Cycle
SRS	Software Requirements Specification
UML	Unified Modeling Language
CI/CD	Continuous Integration / Continuous Delivery
API	Application Programming Interface
BSTI	Bangladesh Standards and Testing Institution
BFSA	Bangladesh Food Safety Authority
DNCRP	Directorate of National Consumer Rights Protection
E2E	End-to-end (test)
FR	Functional requirement
p95	95th-percentile response time
PR	Pull request
RTM	Requirements traceability matrix

Chapter 1
Introduction

1.1 Problem and Motivation

Food and fuel adulteration is a daily risk for people in Dhaka. Street food, restaurant meals and the fuel sold at filling stations can be mixed with cheaper or unsafe substances, sold in short quantity or stored badly. In the 2024-25 fiscal year the Bangladesh Food Safety Authority (BFSA) found that 571 of 1,713 tested food samples, or 33%, were adulterated, contaminated or below standard [4]. Mobile courts regularly fine filling stations for adulterated or short-measured fuel, but these actions are reported days later in the news, after customers have already bought the product.

Consumers who notice a problem today can call the BFSA hotline 16155 or file a complaint with the Directorate of National Consumer Rights Protection (DNCRP) through the hotline 16121 or the MyGov portal [5]. These channels are useful for enforcement, but the complaints are private: other shoppers never learn about them. Public review platforms such as Google Maps are open to everyone, but they are not about safety, and a single fake review can damage an honest shop.

The target users are shoppers and drivers in Dhaka, the moderators who check reports, and the authorities (BSTI, DNCRP and accredited laboratories) that inspect vendors. This problem was chosen because it affects almost everyone in the city, because the information already exists in people's experience but is scattered, and because a software platform is the natural way to collect, verify and publish it at low cost.

1.2 Objectives and Scope

The aim of AwarenessAlert is to turn individual safety concerns into a fair, verified and public warning system. The measurable objectives are:

- O1 — A shopper can report a concern in under two minutes without creating an account.
- O2 — A vendor's trust status changes only after at least three independent sources within 30 days, or one authority confirmation.
- O3 — Verified reports are routed to the responsible authority, whose outcome (Confirmed or Cleared) is shown publicly.
- O4 — The trust status of every vendor is visible on a public map together with the reasons behind it.
- O5 — Reporters stay private: IP address and device ID are never shown to the public or to authorities.
- O6 — Every functional requirement of the SRS is traced to at least one passing automated test.

Scope: the product covers the public website (map, search, vendor profiles, recent reports, reviews, saved areas), the admin portal and the authority portal for the pilot area Gulshan–

Banani. It does not include a live government data connection, a vendor dashboard, a mobile app or SMS delivery, which are future work. Assumptions: users have internet access, authorities are willing to use the portal, and the vendor list is seeded by administrators.

1.3 Team Organization

This is an individual project. I, Zearin Akhter, was responsible for every part: requirements and design, the React front end, the PHP API and database, testing, and all documentation. Work was planned around the weekly course experiments and tracked with a personal checklist and Git commits, following an iterative process [1], [2]; the testing work was done on a separate branch and integrated through a reviewed pull request. The contribution record is in Appendix B.

Chapter 2
Requirements and Design

2.1 Existing Solutions and Alternatives

Table 2.1 compares AwarenessAlert with two existing solutions. The BFSA hotline 16155 is the official channel for food complaints, and Google Maps reviews are the most widely used public opinion platform. The DNCRP channel (hotline 16121 and MyGov online complaints) works like the BFSA hotline: complaints are private and handled one by one [5].

Table 2.1. Comparison with existing solutions

Feature	BFSA hotline 16155	Google Maps reviews	AwarenessAlert
Warning visible to the public	No	Yes	Yes
Focus on food and fuel safety	Food only	No (general reviews)	Yes
Protection against fake reports	Officer checks each call	No	3-source rule + moderation
Official outcome shown publicly	No	No	Yes (Confirmed / Cleared)
Reporter privacy	Phone number known	Public profile	Anonymous; IP admin-only
Cost / platform	Free, phone call	Free, app and web	Free, web

For the technology, three alternatives were considered. Laravel would have given more structure but adds a framework I had not used; a Node.js and MongoDB stack would need new skills and gives up the foreign keys that the report–vendor–review data needs. React with TypeScript and a plain PHP API on MySQL was chosen because it runs on XAMPP, which I already use, costs nothing to host, and keeps the relational data consistent.

2.2 Requirements Summary

The stakeholders are shoppers and drivers (guests and registered users), vendors (shops and filling stations), moderators and administrators, authority organizations (BSTI, DNCRP, accredited laboratories) and the course instructor as supervisor. The SRS [Appendix C] defines 66 functional requirements in 11 features and the non-functional requirements of sections 3 and 5.

Key functional requirements. Guests can report without an account, with a category, vendor, location, a description of at least 15 characters and an optional JPG or PNG photo up to 10 MB (FR-2). Moderators verify, dispute, re-open, remove and escalate reports, and every action is logged (FR-3). A vendor’s status changes only when at least three independent sources report within 30 days or an authority confirms (FR-4, BR-1). Escalated reports are routed by category to BSTI, DNCRP or a laboratory, which records the outcome with a public comment of at most 280 characters (FR-8).

Key non-functional requirements. Public pages load within 3 seconds and the API answers within 1 second (PR-1); passwords are stored as one-way hashes and every protected request is checked on the server (SE-1, SE-3); IP addresses and device IDs are visible only to admins (SE-5, QA-6); the interface should be usable on mobile phones and meet WCAG 2.2 AA checks [7] (QA-1).

User story 1. As a shopper, I want to report unsafe food without an account, so that other people are warned. Acceptance: given the report form, when I submit a valid report, then I receive a report number such as AA-01048 and the report is shown as “Under review”.

User story 2. As a vendor, I want my status to stay the same after a single complaint, so that a competitor cannot damage my business. Acceptance: given a Trusted vendor, when one verified report arrives, then the status stays Trusted until three independent sources or one authority confirmation exist.

2.3 System Design

AwarenessAlert has a three-layer architecture (Fig. 2.1). The React 19 and TypeScript single-page application provides the public site, the admin portal and the authority portal. It talks to a PHP REST API through JSON requests. The API handles sessions, checks the user’s role on every request, validates input and enforces the moderation state machine, and stores data in MySQL or MariaDB through PDO prepared statements. When the database is offline, the front end switches to a demo mode that keeps data in browser storage, so the presentation never depends on the server.

The key models are the use-case diagram with five actors (Fig. 2.2), the ER diagram of the database (Fig. 2.3) and the sequence diagram of the main feature, a report that moves from submission to an official outcome (Fig. 2.4). The class, activity and DFD models are in the design document [Appendix C]. The main screens are shown in Section 4.5.

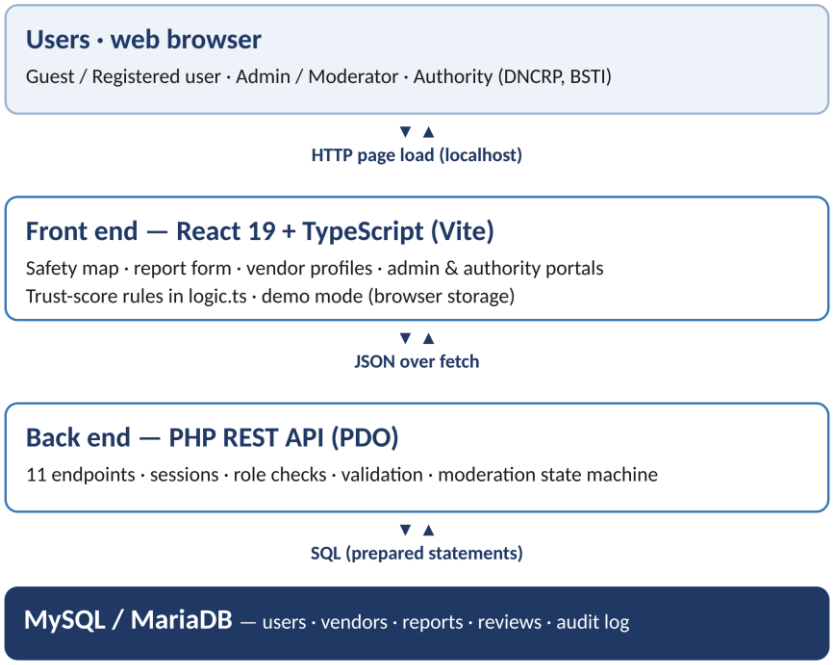


Fig. 2.1. System architecture

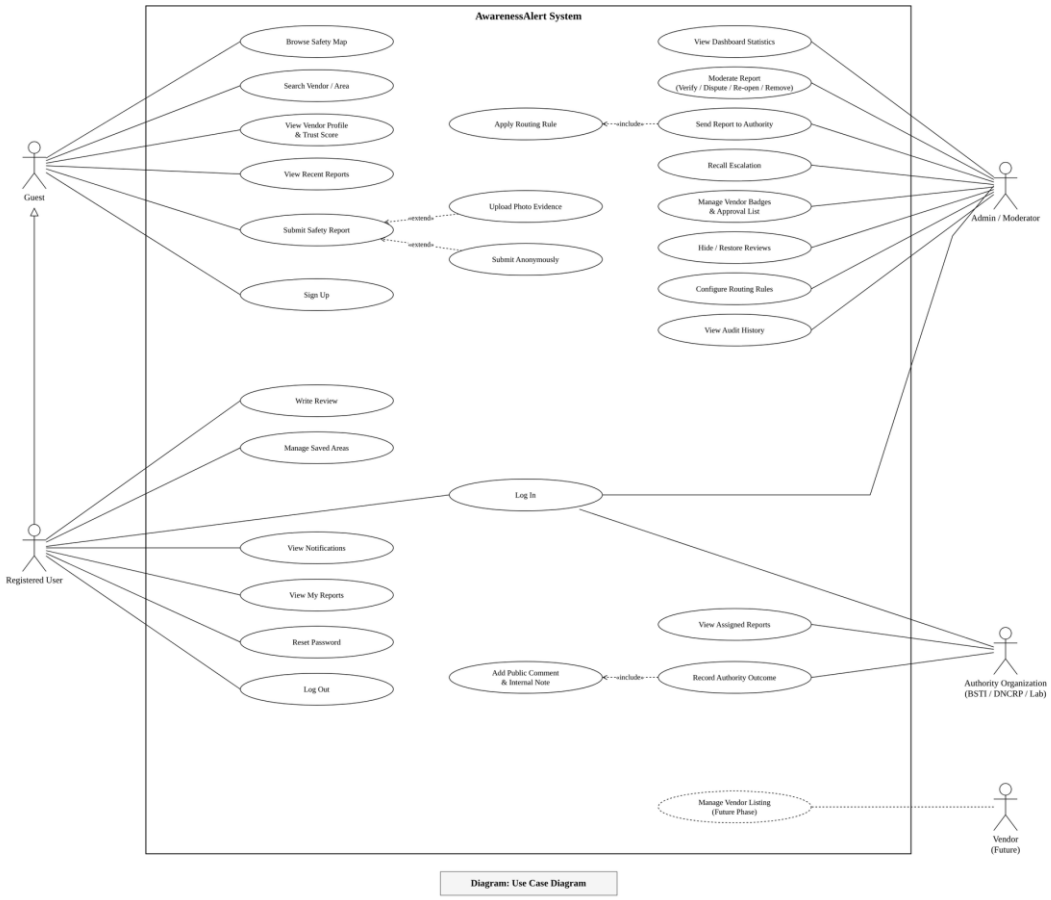


Fig. 2.2. Use case diagram

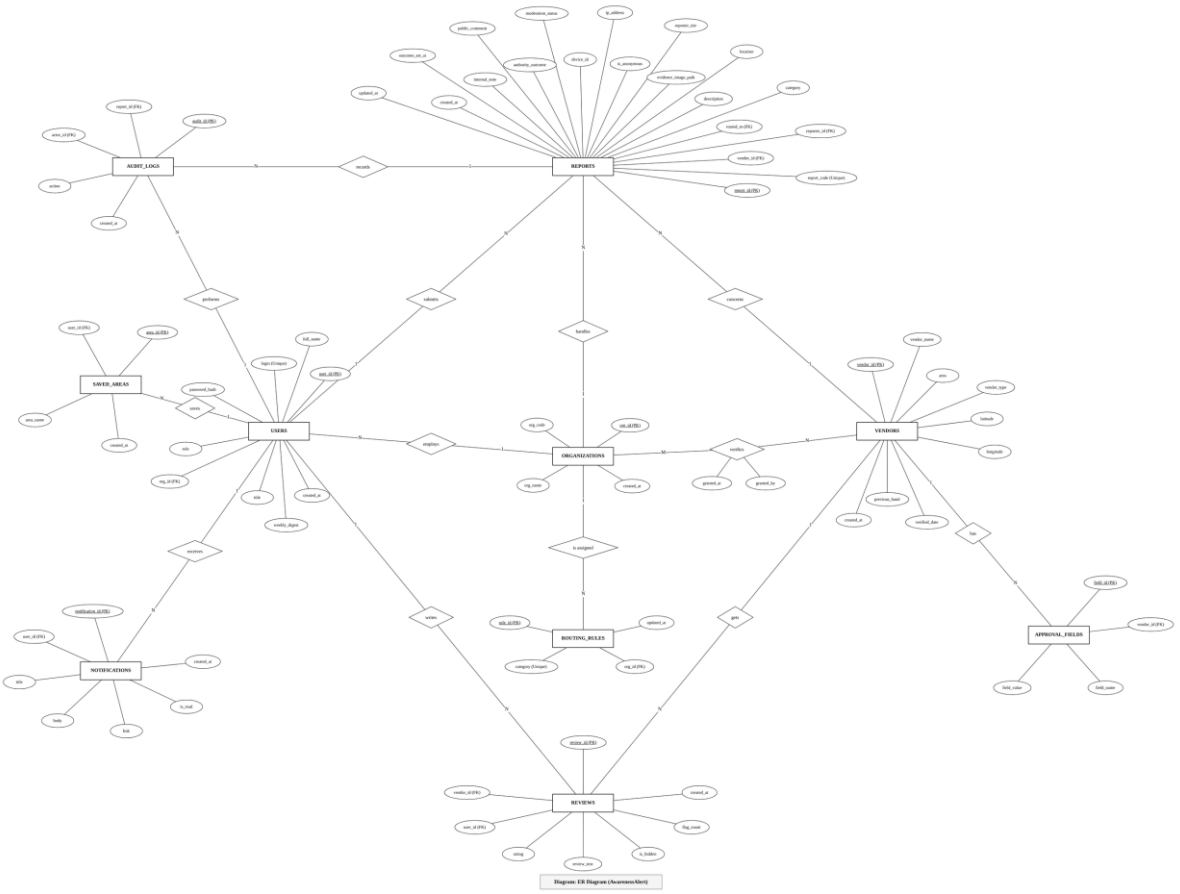


Fig. 2.3. Entity-relationship diagram

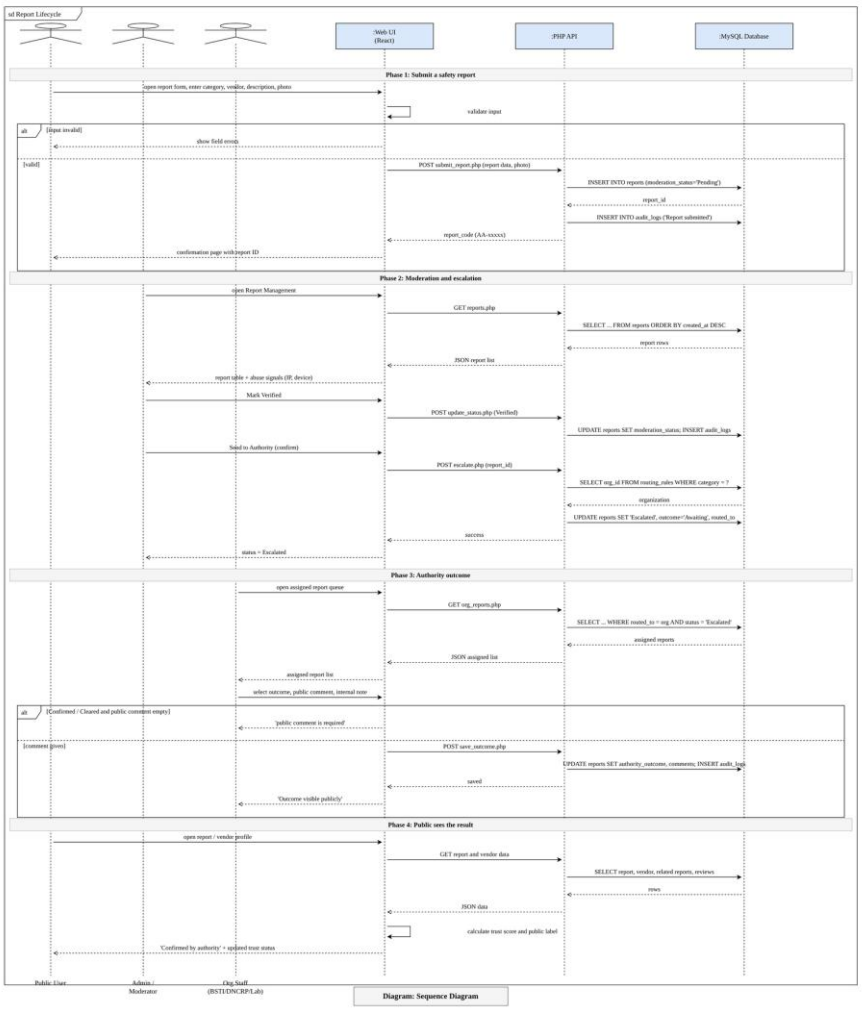


Fig. 2.4. Sequence diagram: report to authority outcome

2.4 Design Decisions and Rationale

D1 — Trust rule. Alternatives: change the status on every report, or show a star average. Chosen: the status changes only with at least three independent sources in 30 days or one authority confirmation; then the score is 92 minus the weight of the reports, never below 18. Evidence: this protects honest vendors from one fake complaint (BR-1), and unit tests TC-U01–U07 check the boundaries (2 versus 3 sources, 30 days versus 30 days and one minute).

D2 — Business rules on the server. Alternative: enforce rules only by hiding buttons in React. Chosen: every rule is checked again in PHP. Evidence: testing found that hidden reviews were still returned by the API (D-05) and that an admin could recall a report already confirmed by an authority (D-09); both were fixed on the server.

D3 — Technology stack. Alternatives: Laravel; Node.js with MongoDB. Chosen: React + TypeScript, plain PHP with PDO, MySQL. Evidence: zero hosting cost, runs on XAMPP, relational integrity with foreign keys, and skills I already had.

D4 — Privacy by design. Alternative: show the reporter's name to authorities. Chosen: anonymous reporting by default; IP and device ID only for admins. Evidence: protects reporters from retaliation while still letting moderators detect abuse (FR-5).

Chapter 3
Implementation

3.1 Implementation and Integration

The development environment was Windows 11 with VS Code, XAMPP (Apache, PHP and MariaDB), Node.js and Vite. The project scaffold (React, Vite, Tailwind CSS) was created in Figma Make and then developed further by hand. The code is kept in Git: the main branch holds the working product, and testing and fixes were done on a separate testing branch merged through a pull request that Qodo reviewed automatically.

Core modules. The front end is in src/: App.tsx contains the pages and portals, logic.ts holds the trust-score rules as pure functions, and api.ts is the API client with the demo-mode fallback. The back end in Backend/awarenessalert-api/ has one PHP file per resource: auth.php (login, sign-up, sessions), reports.php, report_action.php (moderation and escalation), reviews.php, users.php, vendors.php, login_history.php and data.php, with db.php for the PDO connection and setup.php / upgrade.php for the schema. All modules were built by me.

The trust rule. The excerpt below from logic.ts decides whether a vendor's status may change:

```
const sources = new Set(reports
  .filter(r => at - new Date(r.submittedAt).getTime() <= 30 * DAY)
  .map(r => r.reporterId ?? r.deviceId)).size
const threshold = sources >= 3 || confirmed > 0
if (threshold) {
  score = Math.max(18, 92 - weighted)
  band = score >= 70 ? "Trusted" : score >= 40 ? "Caution" : "High risk"
}
```

Reports are counted by reporter account or, for guests, by device, so three reports from the same phone count as one source. Only when the threshold is met is the new score calculated; otherwise the previous band stays.

Integration. The interface was connected to the API screen by screen: first the public map and report form, then the admin portal and finally the authority portal. One integration defect: a four-letter Bangla password was accepted although the rule requires eight characters, because PHP strlen() counts bytes and each Bangla letter takes three bytes in UTF-8 (D-02). The fix was mb_strlen() in auth.php and users.php, with regression test IT-13.

3.2 Tool-Assisted Refinement

Staff validation (code review). I wrote the staff form validation for the Team page first. Qodo reviewed pull request #1 and reported a high-severity bug: the new validateStaffInput function was never called and allowed six-character passwords. I accepted the finding and fixed it, but rejected Qodo's own summary, which described six characters as the intended rule: the SRS (FR-1.1) requires eight. The tests TC-U31 and IT-62 now check the SRS value.

Test suite (testing). I designed the test cases from the SRS rules first: equivalence partitions, boundary values and the moderation state transitions. Claude was then used to set up Vitest, Playwright and the CI file and to write test code for these cases. I verified the result by planting three bugs by hand and by mutation testing (86.9% of mutants killed); every expected value

was checked against the SRS. Diffblue Cover was also considered but supports only Java, so it could not be used.

Chapter 4
Testing, Deployment and Project Management

4.1 Testing and Quality Assurance

The Software Test Report v1.0 [Appendix C] tested tag v0.9-test (commit 01fec35) at the Professional tier of the course guideline, using the ISO/IEC/IEEE 29119-3 document structure [6]. Three levels were used: 58 unit tests (Vitest) on the trust rules and API client, 83 integration tests that call every PHP endpoint against a separate MariaDB test database, and 78 end-to-end runs (Playwright) on desktop Chrome and a Pixel 7 phone, including axe accessibility checks. Techniques were equivalence partitioning, boundary value analysis, decision tables and state-transition testing. Testing found 16 defects; 13 were fixed with a regression test each, and three minor ones are open. Table 4.1 shows a sample of the traceability; the full matrix is in docs/testing/RTM.md.

Table 4.1. Requirements traceability: SRS to design, code and tests

Req ID	Requirement (short)	Design element	Module / file	Test case IDs	Status
FR-1.1	Sign-up with password of at least 8 characters	Sign up use case; users table	auth.php	IT-10–IT-14, E2E-11	Pass
FR-2.1–2.8	Report without an account	Report a concern use case	reports.php; App.tsx	IT-20–IT-27, E2E-01	Pass
FR-3.2–3.7	Moderation state machine	Sequence diagram (Fig. 2.4)	report_action.php	IT-100–IT-105, E2E-19	Pass (D-09 fixed)
FR-4.1–4.7	Trust status needs 3 sources or authority	Trust rule (D1)	logic.ts	TC-U01–TC-U20	Pass
FR-5.1–5.5	Abuse prevention, privacy	reports table (IP, device)	reports.php; data.php	IT-30, IT-125, TC-U04	Pass; FR-5.4 open (D-06)
FR-8.1–8.7	Authority records outcome	Authority use case	report_action.php	IT-50–IT-54, E2E-19	Pass
FR-9.4	Hidden reviews not public	reviews table	reviews.php	IT-75	Pass (D-05 fixed)
PR-1	API answers within 1 s	Architecture	tests/performance/p95.mjs	PT-01–PT-03	Pass (p95 21–37 ms)

Table 4.2. Test and quality summary

Measure	Target	Achieved	Met?
Tests passed / total	100% of priority-1	219 / 219 (100%)	Yes
Line coverage (core logic)	≥ 75%	100% (logic.ts, api.ts)	Yes
Open S1 / S2 defects	0 / 0	0 / 0	Yes
Static-analysis quality gate	Passed	TypeScript strict: 0 errors; SonarCloud not run	Partly
Dependency vulnerabilities (Critical / High)	0 / 0	0 / 0 (npm audit)	Yes

4.2 Code Review, Security and Ethics

I reviewed my own code against the SRS while writing tests, and Qodo reviewed pull request #1 (one high-severity finding, fixed). npm audit reported no vulnerable dependencies after upgrading Vitest, and gitleaks found no secrets in the Git history. A manual check against the OWASP Top 10 [3] confirmed role checks on every protected endpoint, prepared statements against SQL injection, escaped output against cross-site scripting and bcrypt password hashes; missing rate limiting and login lock-out (D-06, D-07) remain open.

The main third-party components are React, Vite, Tailwind CSS and Vitest (MIT licence), Playwright (Apache-2.0), PHP (PHP License) and MariaDB (GPL v2). They are used unmodified and their licence files are kept in the dependencies. The product stores account e-mail or phone numbers, hashed passwords, report texts and photos, and the IP address and device ID of each report; the last two are shown only to admins and never to the public or authorities. Database settings are read from environment variables, so no production secret is in the repository; the demo passwords in the README belong to test accounts only.

4.3 Documentation, CI/CD and Deployment

The documentation consists of the SRS, the design diagrams, the README (installation, demo accounts, running the tests), the Software Test Report and this report. Work on testing was done on the testing branch and merged through pull requests; defects were recorded in the test report's defect log.

The CI pipeline in .github/workflows/test.yml (GitHub Actions [8]) runs on every push and pull request: it installs dependencies, type-checks the code, runs the unit tests with coverage and runs npm audit and gitleaks. A failing test makes the run fail, so the change is not merged or released. Integration and end-to-end jobs are in the same file and run on demand, because their database and browser setup on the GitHub runner still needs tuning; they pass locally. The submitted version is tagged v0.9-test. The product is deployed on InfinityFree free hosting (PHP 8.4, MySQL) at https://awarenessalert.infinityfreeapp.com. The built React files and the PHP API share one domain, so no cross-origin settings are needed; database settings live in a config.php that exists only on the server. A deploy job in the same workflow uploads the site by FTP after the unit and security jobs pass on main; the first release was uploaded through the host's File Manager. During deployment the host ignored putenv(), which left the API on the XAMPP defaults; db.php was changed to read the settings from config.php directly, and a temporary checker page confirmed the connection.

4.4 Project Management

Table 4.3 compares the plan with the actual progress. Everything was finished on time; deployment was completed in week 13.

Table 4.3. Planned versus actual schedule and key risks

Milestone / risk	Planned (week)	Actual (week)	Variance, reason or mitigation
Problem selection and SRS	1–2	1–2	On time (SRS v2 submitted)

Milestone / risk	Planned (week)	Actual (week)	Variance, reason or mitigation
Design models	3–4	3–4	On time (UML, ER, DFD in draw.io)
Implementation and integration	5–8	5–8	On time; mid-term demo held
Testing and test report	9–10	9–10	On time; 16 defects found, 13 fixed
Deployment and presentation	12–13	13	Deployed on InfinityFree; host blocked putenv, fixed in db.php
Risk: fake or malicious reports	—	—	3-source rule, moderation and audit log; in place
Risk: database down during demo	—	—	Demo mode with browser storage; in place
Risk: solo workload	—	—	Weekly plan per experiment; reused course templates

Estimated effort and cost: 1 member × about 160 hours over 14 weeks; hosting cost BDT 0 (InfinityFree free plan).

4.5 Results and Discussion

AwarenessAlert meets all six of its objectives (Section 5.1): reporting without an account works, the three-source rule protects vendors, reports are routed to authorities with public outcomes, the map explains every status, and reporter data stays private; all 66 functional requirements pass their tests except FR-5.4 (rate limiting), which is documented as open defect D-06. Compared with Table 2.1, it is the only option that combines public warnings with protection against fake reports and an official outcome. After the mid-term review, the remaining work listed in the SRS — connecting all screens to the PHP back end — was completed. Figs. 4.1–4.4 show the key screens.

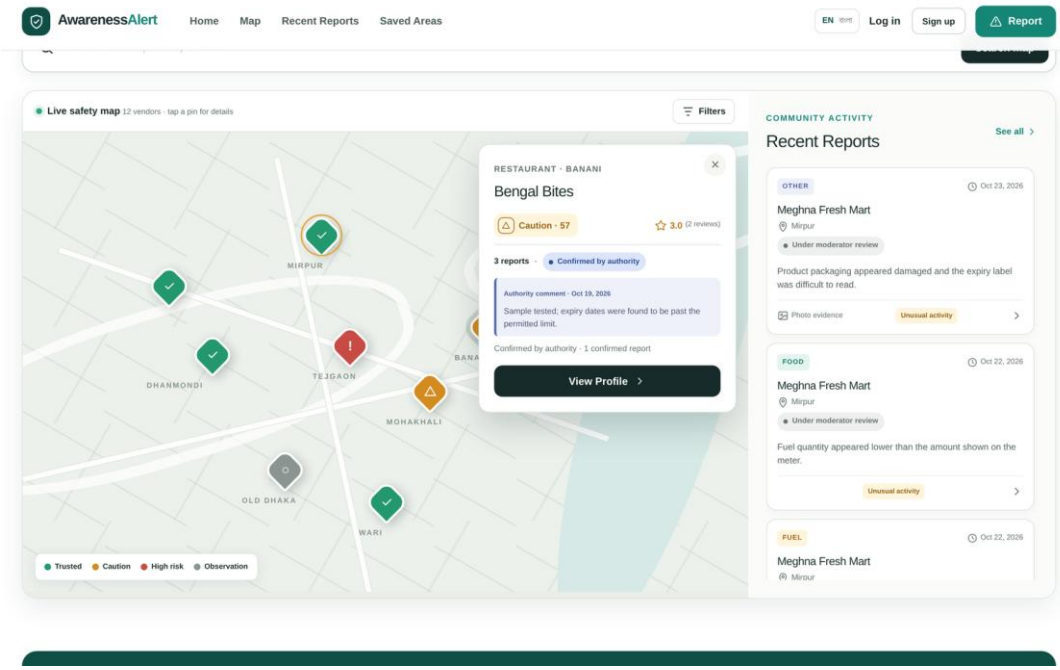


Fig. 4.1. Public safety map

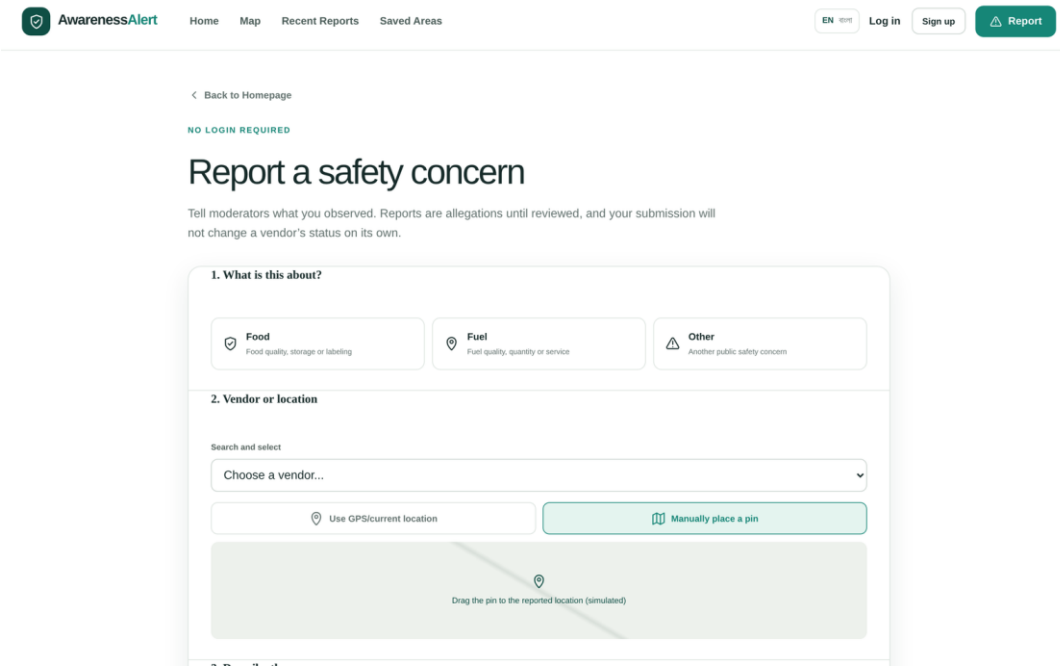


Fig. 4.2. Report form

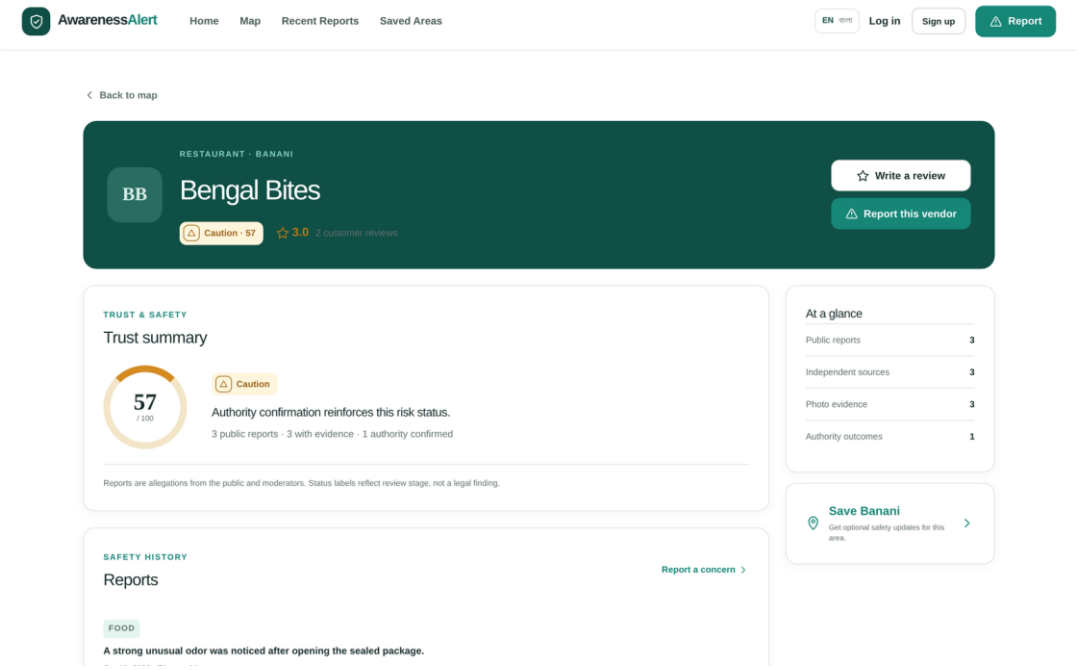


Fig. 4.3. Vendor profile with explained trust score

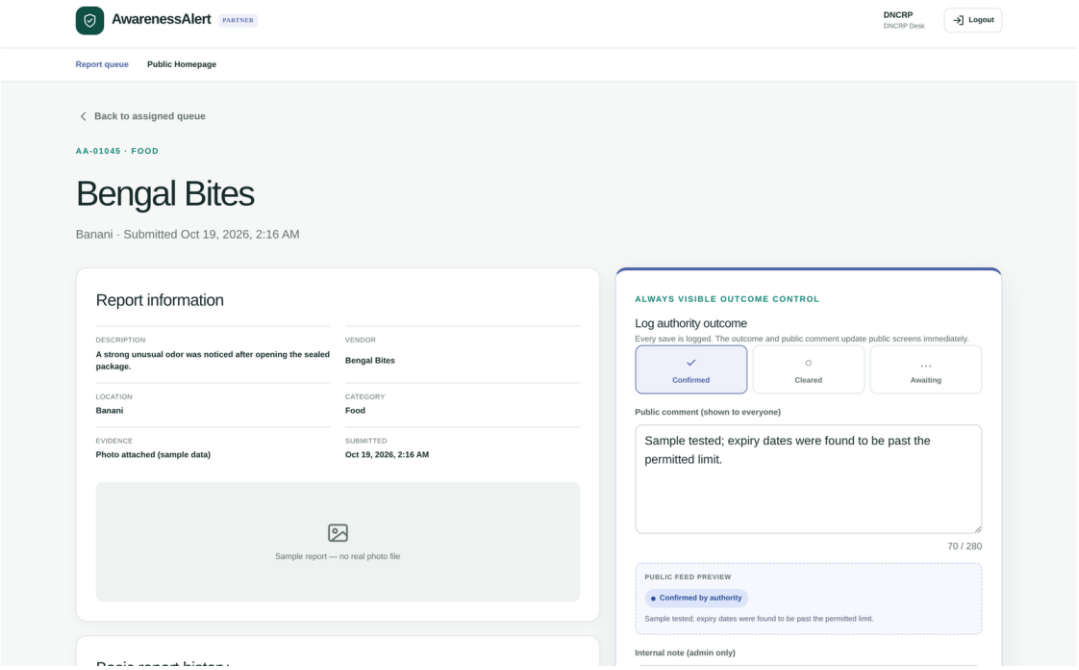


Fig. 4.4. Authority portal: recording an outcome

Chapter 5
Conclusion

5.1 Conclusion and Future Work

AwarenessAlert delivers a working platform that turns private safety concerns into fair public warnings. Against the objectives of Section 1.2: O1 is met (guest report with report number, tested end-to-end); O2 is met (trust rule in logic.ts with 100% coverage); O3 is met (routing rules and authority portal); O4 is met (map pins and explained vendor profiles); O5 is met (anonymous reports, admin-only IP and device ID); O6 is met (66 of 66 functional requirements traced to passing tests). The product is also deployed online at https://awarenessalert.infinityfreeapp.com.

Future work: move to paid hosting with its own domain; add rate limits and login lock-out; add a Bangla interface; send e-mail or SMS alerts and the weekly digest; build the vendor dashboard so that shops can claim their listing and reply; run a usability test with real users; and connect to official data when authorities offer an API.

5.2 Reflection

Designing first made the work easier to check. Because the SRS already defined the rules, I could judge every tool output against it. The clearest example was Qodo: its summary of pull request #1 described a six-character password as correct because it read my new code, not my requirements. A test generated from that summary would have locked the bug in. Testing also showed that my riskiest code was not the user interface but the server rules: several checks existed only in React. As a solo developer I learned to plan around the weekly experiments, and I learned that ethical responsibility includes protecting reporters' privacy, being fair to vendors, and openly disclosing how much an AI assistant helped with the tests and the documents.

References

[1] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York, NY, USA: McGraw-Hill, 2020.

[2] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson, 2015.

[3] OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: https://owasp.org/Top10/

[4] "33% food samples tested unsafe in FY 2024-25," The Daily Star, Feb. 2, 2026. [Online]. Available: https://www.thedailystar.net/business/news/33-food-samples-tested-unsafe-fy-2024-25-4096726

[5] "Bangladesh National Consumer Rights Act: How to complain against faulty goods, services," UNB. [Online]. Available: https://unb.com.bd/category/lifestyle/bangladesh-national-consumer-rights-act-how-to-complain-against-faulty-goods-services/94356

[6] ISO/IEC/IEEE 29119-3:2021, Software and Systems Engineering — Software Testing — Part 3: Test Documentation. Geneva, Switzerland: ISO, 2021.

[7] W3C, "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation, Oct. 2023.

[8] GitHub, "GitHub Actions documentation," GitHub Docs. [Online]. Available: https://docs.github.com/actions

Appendix A: Tool Usage Log

Every use of an assistant tool is recorded below. In each case I produced my own design first.

Table A.1. Tool usage log

Wk	Task / artifact	Tool used	Own design first (what we did)	Tool's role (refine / explore / verify)	How the output was verified	Decision (accepted / modified / rejected)
2–4	SRS, design diagrams	draw.io (no AI)	All requirements and models written by me	—	—	—
5–7	Project scaffold, first UI screens	Figma Make	Feature list and screens from my SRS	Explore: generated the React/Vite/Tailwind starting code	Ran and checked every screen against the SRS	Modified
9	PR #1 staff validation	Qodo PR review	Validation function written by me	Verify: code review	Compared with SRS FR-1.1 (8 characters)	Finding accepted; summary rejected
9	Unit-test generation	Diffblue Cover	Test cases from SRS	—	—	Rejected: Java only
9–10	Test tooling and test code	Claude	Test cases, partitions and expected values from my SRS	Refine: wrote test code, CI file	Planted bugs; mutation score 86.9%; all values checked against SRS	Accepted after review
10–13	Test report, final report, slides	Claude	Content, numbers and structure from my SRS, code and test runs	Refine: drafted text in the course templates	Every figure checked against the test evidence	Modified

Appendix B: Team Contribution and Peer Evaluation

This is an individual project, so all work was done by one member, as the commit and pull-request history shows.

Repository: https://github.com/Zearin14/Zearin_CSE-416_SDD_Project-_AwarenessAlert

Deployed product: https://awarenessalert.infinityfreeapp.com

Table B.1. Contribution summary

Member (name, ID)	Modules / tasks owned	Commits / pull requests	Documentation, testing and review role
Zearin Akhter (0432320005101014)	All modules: React UI, PHP API, database, trust logic	All commits; PR #1 (reviewed by Qodo) and the testing branch	All documentation, all testing and review

Table B.2. Agreed contribution split

Member (name, ID)	Agreed contribution (%)	Signature	Date
Zearin Akhter (0432320005101014)	100%	Zearin	31-10-2026

Individual project: one member, 100% of the work.

Table B.3. Peer evaluation (each member rates the other: 4 = Excellent, 1 = Needs work)

Criterion	Not applicable	Not applicable
Technical contribution	N/A	N/A
Reliability and meeting deadlines	N/A	N/A
Communication and collaboration	N/A	N/A
Code review and quality of work	N/A	N/A
Comment	Individual project — no peer evaluation	N/A

Appendix C: Supporting Documents

The following documents were submitted during the semester and are attached or linked:

- **Software Requirements Specification: AwarenessAlert SRS Version 2.0 (Experiment 2).**
- **Software Test Report: Experiment 9 report v1.0, version tested v0.9-test (commit 01fec35); also in the repository at docs/testing/Test_Report.pdf.**
- **Code-review and security log: Section 4.2 and §8.2 of the Software Test Report (npm audit, gitleaks, OWASP checklist, Qodo review).**
- **README / user guide: https://github.com/Zearin14/Zearin_CSE-416_SDD_Project-_AwarenessAlert#readme (installation, demo accounts, running the tests).**

Evaluation Sheet (Instructor Use Only)

Table E.1. Report evaluation

Criterion	Evidence in report	CO	Marks	Member 1	Member 2
Problem analysis, SRS and design	Ch. 2; Table 4.1	CO1	15		
Investigation and design rationale	2.1, 2.4	CO5	10		
Implementation and use of modern tools	Ch. 3; App. A	CO2	10		
Testing and quality assurance	4.1–4.2; App. C	CO3	15		
Deployment, CI/CD and project management	4.3–4.4	CO4	15		
Professional and ethical responsibility	4.2; AI Ack.; App. A	CO8	10		
Teamwork and individual contribution	1.3; App. B	CO7	10		
Communication: structure, clarity, figures, references	Whole report; App. C	CO6	15		
Total			100		

Instructor: